

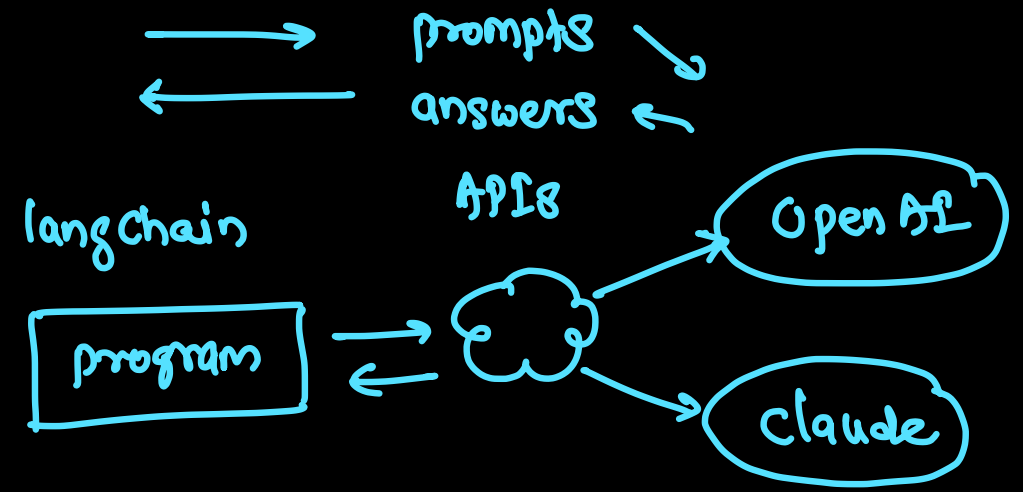


Build Your Own LLM From Scratch

From First Principles — No Magic, No APIs

What AI really today ?

- LLMs feel like black box
- Everyone is using ChatGPT
- Very few understand how it works
- Almost no one can build one
- Fear of math blocks
- Huge gap between demo and production
- Short term skills without long term benefits



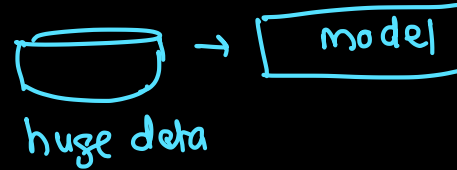
What is Language Model ?

hello → world



- A language model is a system that learns patterns in human language so it can understand, predict, and generate text
 - numeric data
 - prediction
- In short: Given some text, it predicts what comes next
- It does not truly “understand” like humans or does not know facts inherently
- A language model is a probability machine trained on text that predicts and generates language by learning patterns in how words appear together
- What do they do ?
 - Text generation
 - Question answering
 - Translation
 - Summarization
 - Chatbots
 - Code completion
 - Reasoning (to some extent)

hello hell





Types of language models

■ Statistical language models

- N-gram models
- Based on word frequency counts
- Limited context window
- Example: predicting using last 2 – 3 words only

■ Neural language models → RNN → LSTM → GRU

- Use neural networks
- Learn word meanings (embeddings)
- Can generalize better than n-grams

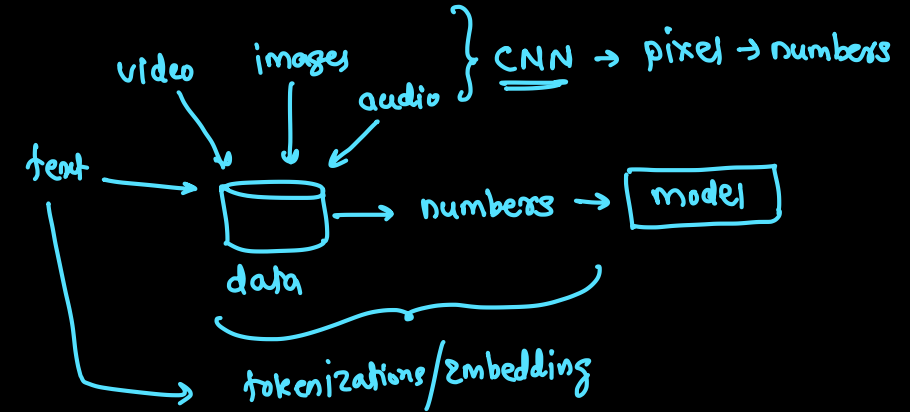
■ Transformer-based language models (modern LMs)

- Use attention mechanism
- Handle long context
- Power today's AI assistants
- Example capabilities:
 - Answer questions
 - Summarize text
 - Translate languages
 - Write code

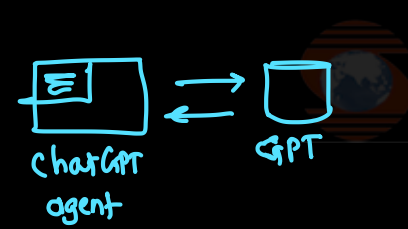
What is Large Language Model (LLM)?



- A Large Language Model (LLM) is a language model trained on massive amounts of text with a very large neural network, enabling it to perform many language tasks without task-specific training
- Large refers to:
 - Model size (parameters: millions → billions → trillions)
 - Training data size (books, code, web, papers)
 - Compute power (GPUs/TPUs, distributed training)
- **Key characteristics of LLMs**
 - **Transformer-based**
 - Almost all LLMs use the Transformer architecture
 - Heavy use of self-attention to capture long-range context
 - **Pre-trained, then adapted**
 - First trained on general text (pretraining) → foundational model
 - Later adapted using Fine-tuning, Instruction tuning and Reinforcement Learning from Human Feedback (RLHF)
 - **General-purpose**
 - One single model can Answer questions, Write essays, Generate code, Translate languages, Summarize documents
- Examples: GPT series, BERT, LLaMA, Gemini, Claude



What LLM is not?

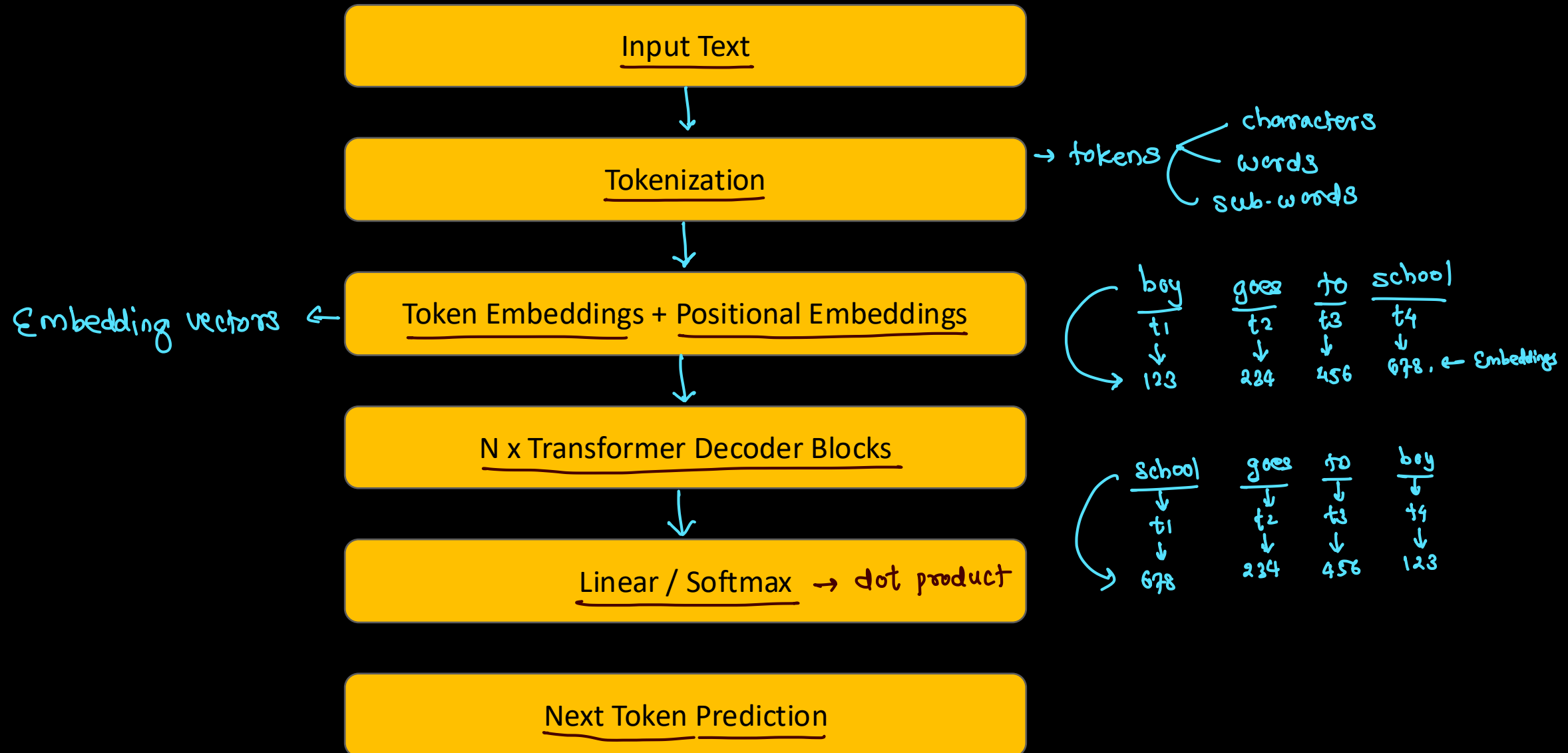


- **Not an intelligent human**
 - LLMs do not think, feel, or understand like humans
- **Not a source of absolute truth**
 - LLMs do not know facts — they reproduce patterns
 - Can confidently give wrong answers (hallucinations)
 - No built-in fact-checking
 - Knowledge may be outdated
- **Not a reasoning engine (by default)**
 - LLMs don't reason like logic engines or humans
 - Reasoning is pattern imitation
 - Can fail at Multi-step logic, Math proofs, Symbolic reasoning
 - Appears to reason because such patterns exist in training data
- **Not a database or search engine**
 - LLMs do not retrieve documents unless connected to tools.
 - No live internet access by default
 - Cannot “look up” new information
 - Answers come from **model weights**, not queries
- **Not a deterministic system**
 - Same input ≠ same output (always).
 - Uses probability sampling
 - Output varies with Temperature and Top-k / Top-p
 - Not guaranteed repeatability unless constrained
- **Not a memory system**
 - LLMs do not remember past conversations unless explicitly designed to.
 - No long-term memory by default
 - Each prompt is stateless “Memory” must be engineered externally
- **Not a world model with grounding**
 - LLMs don't have real-world experience.
 - Never sees, hears, or acts (unless multimodal extensions)
 - No physical intuition
 - No causal understanding beyond text patterns
- **Not a replacement for domain experts**
 - LLMs **augment** humans, they don't replace expertise.
 - Cannot guarantee correctness
 - Needs human verification
 - Unsafe for high-risk decisions alone (medical, legal, finance)

How does the LLM work ?



training the LLM



How TinyGPT worked?



■ Tokenization → BPE

- Models never see text, it can work with only numbers
- E.g. hello → [7, 4, 11, 11, 14]
- Tokens can be: Characters, Words, Sub-words

cosine

↪ dot product

■ Embeddings

- numerical vector representations of data that capture meaning, similarity, and relationships
- Each number becomes a vector
- Embeddings encode: Semantic meaning, Contextual usage, Relationships, Analogies
- E.g. king - man + woman ≈ queen
- Algorithms: Word2Vec, GloVe, FastText, BERT etc

■ Transformers → Encoder - decoder arch

- Before transformers, models read text one word at a time which was slow, forgetful and impossible to scale
- Transformers are neural network architectures that process sequences using self-attention instead of recurrence, allowing models to understand relationships between all tokens in parallel → multi head attention
- In simple terms: each word looks at every other word to decide what matters most, making Transformers fast, scalable, and powerful for language understanding and generation

How TinyGPT worked?



■ Attention mechanism

- The attention mechanism lets a model focus on the most relevant parts of the input when processing a token
- Not all words are equally important. Attention assigns higher weight to the words that matter most for the current word
- Types of attentions:
 - Self-attention: tokens attend to tokens in the same sequence
 - Masked (causal) attention: prevents looking at future tokens (used in GPT)
 - Cross-attention: one sequence attends to another (used in encoder-decoder models)
- Every token asks:
 - What am I looking for? (Query) → Q
 - What do I contain? (Key) — K
 - What do I give? (Value) — V

✓ Multi-head attention

- Instead of using one attention, Transformers use many attentions (heads) to focus on different aspects of meaning simultaneously
- A single attention head might focus on: Grammar, Subject-verb agreement, Long-range dependencies
- Multiple heads allow the model to learn all of these at once



What have we covered today ?

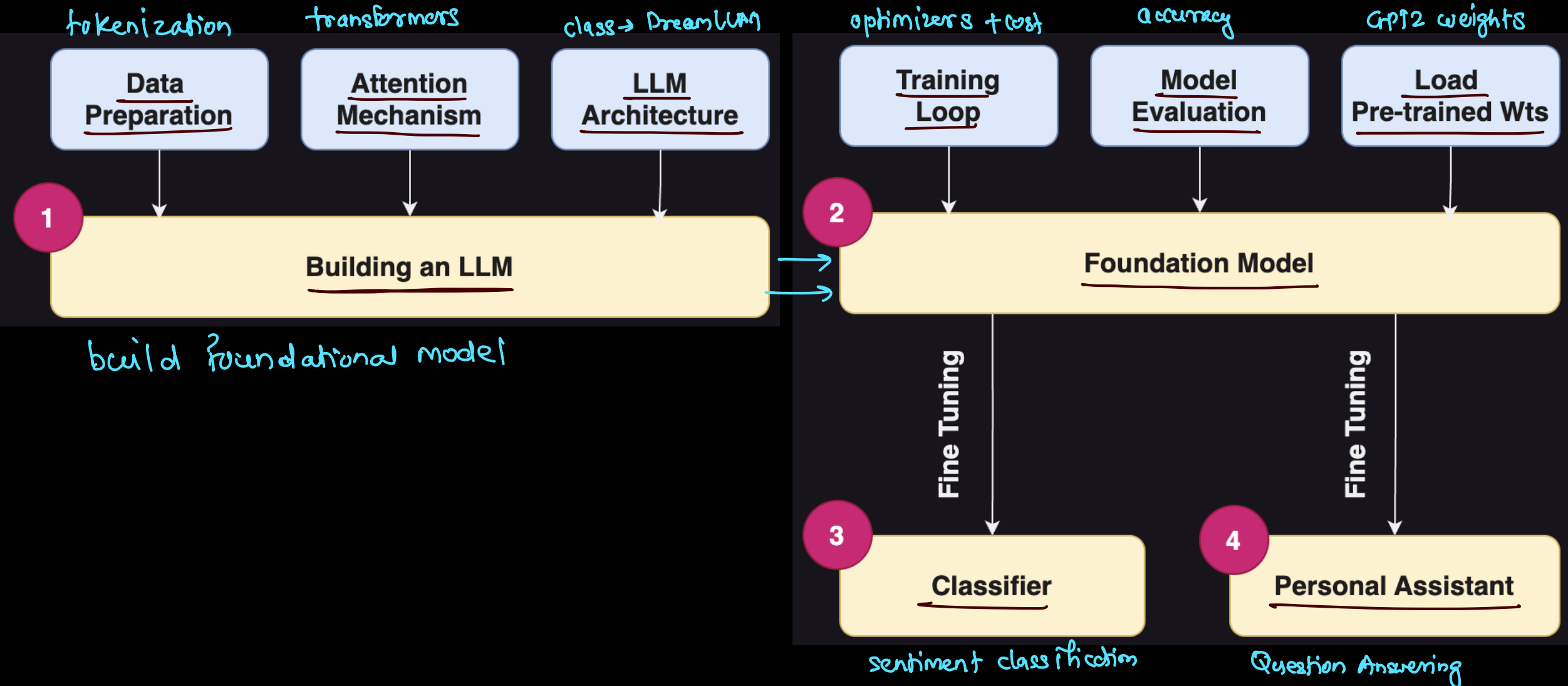
- What is LLM ?
- What an LLM actually does ? (next-token prediction)
- What LLMs are NOT (thinking, reasoning like humans)
- Why LLMs feel intelligent despite being probabilistic
- How ChatGPT, GPT-4, LLaMA all share the same core idea



What we did not cover today

- The math behind transformers: matrices, dot products, softmax, gradients
- Efficient training techniques: optimizers, learning rate schedules, regularization
- Scaling models to millions or billions of parameters: memory, GPUs, distributed training
- Instruction tuning and alignment: why ChatGPT follows instructions but raw GPT doesn't
- Fine-tuning for real-world use cases: domain data, adapters, LoRA and QLoRA
- Retrieval-Augmented Generation (RAG): connecting LLMs to private documents & databases
- Deployment & production concerns: latency, cost, inference optimization, monitoring

How to build LLM from scratch



What are we going to cover in this course ?



- **Foundations: How Language Models Really Work**
 - What language models actually learn (probabilities, not facts)
 - Tokens, vocabularies, and embeddings (from first principles)
 - Why math is needed — and how little you actually need
 - Next-token prediction explained deeply
 - Loss functions and why models fail or repeat
 - Reading LLM outputs like an engineer (not a user)
 - Hand-on: Build different types of tokenizers, vector and matrix operations using PyTorch
 - Outcome: You will understand what an LLM is doing internally — no more black box
- **Transformers from Scratch (Core of LLMs) ✨🔮**
 - Self-attention step by step (Q, K, V intuitively)
 - Positional encodings — why order matters
 - Feed-forward networks inside transformers → f_c
 - Residual connections & layer normalization
 - Encoder vs Decoder (why GPT is decoder-only)
 - Causal masking and autoregressive behavior
 - Hands-on: Build a Transformer block from scratch using PyTorch
 - Outcome: You will be able to read any transformer diagram or paper with confidence



What are we going to cover in this course ?

- Training a Language Model Properly
 - Training loops in detail
 - Backpropagation (conceptual + practical)
 - Optimizers (SGD, Adam, AdamW)
 - Learning rate schedules
 - Why loss explodes or collapses
 - Overfitting, underfitting, and data quality
 - Why small models behave strangely (repetition, nonsense)
 - Hands-on: Train your own mini-GPT and debug its behavior
 - Outcome: You will know why a model behaves badly – and how to fix it
- Scaling, Fine-Tuning & Alignment
 - Why scaling works (data + parameters + compute)
 - Pretraining vs fine-tuning
 - Instruction tuning (why ChatGPT follows instructions)
 - Parameter-efficient fine-tuning (LoRA, adapters – conceptually)
 - When to fine-tune vs when NOT to
 - Common mistakes in fine-tuning
 - Outcome: You will know how real companies customize LLMs



What are we going to cover in this course ?

- Deployment & LLM Engineering Mindset
 - Inference vs training
 - Latency, cost, and memory trade-offs
 - Model quantization & optimization (high level)
 - API vs self-hosted models
 - Monitoring LLM outputs
 - Ethics, safety, and responsible AI
 - How to stay relevant as models evolve
 - Outcome: You will know how to take an LLM from notebook to production.

Programs we will be writing



■ Tokenizer & Text Processing Demos

- Character-Level Tokenizer (From Scratch): Understand the simplest possible tokenizer
- Word-Level Tokenizer & Its Limitations: Why word tokenization fails in real LLMs
- Sub-word Tokenization (BPE – Intuition First): How GPT-style tokenization really works
- Tokenizer Comparison Demo: Same sentence - different tokens - different model behavior
- Out-of-Vocabulary (OOV) Problem Demo: Why tokenizers matter more than you think

■ Embedding & Vector Understanding Demos

- Token and Embedding Visualization Demo: How numbers become meaning
- Embedding Similarity Demo (Cosine Similarity): Why “king – man + woman = queen” works”
- Contextual vs Static Embeddings Demo: Why transformers beat Word2Vec

■ Attention & Transformer Internals Demos

- Single-Head Self-Attention Demo (Step by Step): Manually compute attention scores
- Multi-Head Attention Demo: Why multiple heads capture richer meaning
- Attention Weight Visualization Demo: See what the model is “looking at”
- Positional Encoding Demo: Why order matters in transformers



Programs we will be writing

■ Transformer Architecture Demos

- **Building a Transformer Block from Scratch:** Attention + FFN + Residuals
- **Encoder-Only Transformer Demo (BERT Style):** Why BERT is good at understanding
- **Decoder-Only Transformer Demo (GPT Style):** Why GPT is good at generation
- **Encoder-Decoder Transformer Demo (T5 Style):** Why translation models work differently

■ Language Modelling & Training Demos

- **Next-Token Prediction Demo (Core LLM Task):** The single task behind all LLMs
- **Training a Mini GPT from Scratch:** End-to-end training loop
- **Loss Curve & Overfitting Demo:** How models learn and fail
- **Repetition & Mode Collapse Demo:** Why models output “heeeee...”
- **Temperature, Top-K & Top-P Sampling Demo:** Control creativity vs correctness

■ Fine-Tuning & Adaptation Demos

- **Pretrained vs Fine-Tuned Model Comparison:** Why fine-tuning changes behavior
- **Instruction Tuning Demo:** Why ChatGPT follows instructions
- **Parameter-Efficient Fine-Tuning (LoRA and QLoRA- Conceptual Demo):** Fine-tuning without retraining everything



Programs we will be writing

- **RAG (Retrieval-Augmented Generation) Demos**
 - **Embedding-Based Document Search Demo:** Finding relevant text using vectors
 - **Simple RAG Pipeline Demo:** LLM + your documents
 - **RAG vs Fine-Tuning Comparison Demo:** When to use which
 - **Hallucination Reduction Demo:** Why RAG improves factual accuracy
- **Deployment & Engineering Demos**
 - **Inference vs Training Demo:** Why serving models is different
 - **Latency & Cost Trade-off Demo:** Speed vs quality
 - **Quantization & Model Size Demo (High-Level):** Running LLMs on limited hardware
 - **API-Based vs Self-Hosted LLM Demo:** Control vs convenience
- **Capstone Demo**
 - **Build Your Own ChatGPT-Style System:** Tokenizer + Transformer + RAG + Inference



Pre-requisites

- Basic Programming Knowledge (Required) → python
- Basic Logical & Problem-Solving Skills
- Basic Math Awareness (Not Expertise) → high school math
- NO Prior ML or AI Knowledge Required
- Basic System Setup → No GPU required
- What You Do NOT Need
 - A PhD
 - Advanced math degrees
 - Prior transformer knowledge
 - Prior NLP experience